1) Computer Fundaments

Page No.
Date

AND Gate

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

A	NOT A
0	1
1	0

NAND Gate

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate

A	B	A NOR B
0	0	0
0	1	0
1	0	0
1	1	1

XOR Gate

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

1's complement

5 = 0101

1's complement of 5 = 1010

2's complement of 5 = 1011

* Binary Addition

1) 1010 + 0011

```
 1010
+0011
-----
 1101
```

2) 1111 + 0001

```
 1111
+0001
-----
10000
```

3) 1001 + 1001

```
 1001
 1001
-----
10010
```

4) 0101 + 0110

```
 0101
 0110
-----
 1011
```

* Binary Subtraction

1) 1010 - 0011

2's compliment (0011) = 1101

```
 1010
 1101
-----
*0111
```

1010 - 0011 = 0111

2) 1111 - 1010

1's compliment 1010 = 0101

2's compliment = 0110

```
 1111
+0110
-----
*0101
```

1111 - 1010 = 0101

3) 1000 - 0001

2's compliment = 1111

$$\begin{array}{r} 1000 \\ +\ 1111 \\ \hline 10111 \end{array}$$

1000 - 0001 = 0111

4) 1100 - 0110

1's compliment of 0110 = 1001

2's compliment of 0110 = 1010

$$\begin{array}{r} 1100 \\ 1010 \\ \hline 10110 \end{array}$$

1100 - 0110 = 0110

* Octal Addition

1) $127_8 + 31_8$

$127 = 1 \times 8^2 + 2 \times 8 + 7 = 87$

$31 = 3 \times 8 + 1 = 25$

$$\begin{array}{r} 87 \\ +\ 25 \\ \hline 112 \end{array}$$

8	112	0
8	14	6
	1	1

$127_8 + 31_8 = 160_8$

2) $645_8 + 132_8$

$645 = 6\times64 + 4\times8 + 5 = 421$

$132 = 1\times64 + 3\times8 + 2 = 90$

$$\begin{array}{r} 421 \\ +\ 90 \\ \hline 511 \end{array}$$

8	511	7
8	63	7
	7	7

$\boxed{645_8 + 132_8 = 777}$

3) $204_8 + 574_8$

$204 = 2\times64 + 4 = 132$

$574 = 5\times64 + 7\times8 + 4 = 380$

$$\begin{array}{r} 132 \\ +\ 380 \\ \hline 512 \end{array}$$

8	512	0
8	64	0
8	8	0
	1	1

$\boxed{204_8 + 574_8 = 1000}$

* Octal Subtraction

1) $645 - 127$

$645 = 6\times64 + 4\times8 + 5 = 421$

$127 = 1\times64 + 2\times8 + 7 = 87$

$$\begin{array}{r} 421 \\ -\ 87 \\ \hline 334 \end{array}$$

8	334	6
8	41	1
	5	5

$\boxed{645_8 - 127_8 = 516}$

* Hexadecimal Addition

$(A3)_{16} + 1C_{16}$

$A3 = 10 \times 16 + 3 = 163$

$1C = 1 \times 16 + 12 = 28$

$$\begin{array}{r} 163 \\ 28 \\ \hline 191 \end{array}$$

16	191	15
	11	11

$$\boxed{(AB)_{16} + (1C)_{16} = (BF)_{16}}$$

* Hexadecimal Subtraction

$(C4)_{16} - (3A)_{16}$

$C4 = 12 \times 16 + 4 = 196$

$3A = 3 \times 16 + 10 = 58$

$$\begin{array}{r} 196 \\ - \quad 58 \\ \hline 138 \end{array}$$

16	138	10
	8	8

$$\boxed{(C4)_{16} - (3A)_{16} = (8A)_{16}}$$

CS Code segment
DS Data
SS Stack
ES Extra

Page No.
Date

* 8086 Microprocessor Architecture

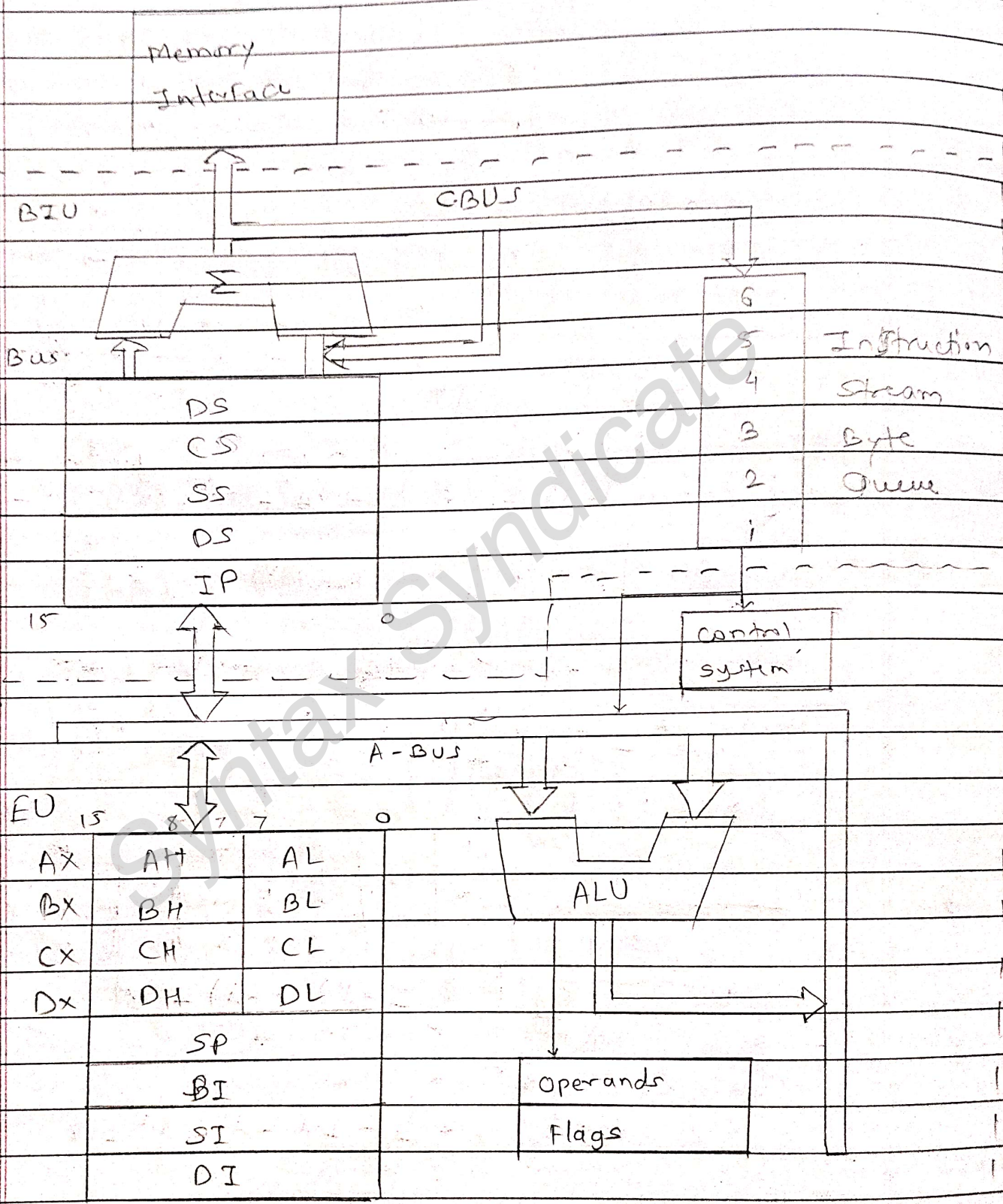
The 8086 is a 16-bit microprocessor, which a 16-bit internal and external data bus, with 20 address lines, it can access upto 2^{20} = 1 MB of memory. The architecture of the 8086 microprocessor consists of two independent section's or units, the Bus Interface Unit (BIU) and Execution Unit (EU).

1) Bus Interface Unit (BIU)

i) Handles data transfer between the processor and memory or I/O.
ii) Responsible for fetching instructions, address generation, and queue management.
iii) Contains: Segment Registers: CS, DS, SS, ES
Instruction Queue: 6-byte prefetch queue Instruction pointer (IP)

2) Execution Unit (EU)

i) Executes the fetched instructions.
ii) Does not directly acess memory.
iii) Contains: Arithemodic logic unit (ALU)
General purpose Registers: AX, BX, CX, DX
Pointer and Index Registers: SP, BP, SI, DI
Flags Register
Control unit
It operate in both minimum and max mode.
It requires +5V power supply.
It has multiplexed address and data bus $AD_0 - AD_{15}$ and $A_{16} - A_{19}$
It can support 64k I/O ports
It can provide 14, 16 bit registers



8086 Internal Architecture.

* 8086 Microprocessor Flags

1) Direction Flag (DF) : If set then string manipulation instructions will auto decrement index registers. If cleared then the registers will be auto incremented.

2) Interrupt - enable Flag (IF) - ~~Selecting~~ Setting this bit enables maskable interrupts.

3) Single - step Flag (TF) - If set then single - step interrupt will occur after the next instruction.

4) Sign Flag (SF) - set if the most significant bit of the result is set.

5) Zero Flag (ZF) - set if the result zero

6) Auxillary Carry Flag (AF) - set if there was a carry from or borrow to bits 0 - 3 in the AL register.

7) Parity Flag (PF) - set if parity (The number of "1" bits) in the low - order byte of the result is even.

8) Carry Flag (CF) - set if there was a carry from or borrow to the most significant bit during last result calculation.

* Addressing Mode

1) Immediate addressing mode.

In this type of addressing, immediate data is a part of instruction, and appears in the form of successive byte or bytes.

eg: MOV Ax, 0005H.

In the above example, 0005H is the immediate data. The immediate data may be 8-bit or 16-bit size.

2) Direct addressing mode.

In the direct addressing mode, a 16-bit memory address (offset) directly specified in instruction as a part of it.

eg - MOV Ax, [5000H].

3) Register addressing mode.

In the register addressing mode, the data is stored in a register and it is referred using the particular register. All the registers, except IP, may be used in this mode.

eg - MOV Bx, Ax

4) Register indirect addressing mode.

Sometimes, the address of the memory location which contain data of operation is determined in an indirect way, using the offset registers the mode of addressing is known as register indirect mode.

In this addressing mode, the offset address of data is in either Bx or SI or DI Register.

5) Indexed addressing mode.

In this addressing mode, offset of the operand is stored one of the index registers. DS, ES are the default segments for index registers SI & DI resp.

eg - MOV Ax, [SI]

Here, data is available at an offset address stored in SI in DS.

6) Register relative addressing mode.

In this addressing mode, the data is available at an effective address formed by, adding 8-bit or 16-bit displacement with content of any one of the register Bx, BP, SI & DI in the default (either in DS & ES) segment.

eg - MOV Ax, 50H [Bx]

7) Based indexed addressing mode.

The effective address of data in formed in this addressing mode, by adding content of a base register (any one of Bx or BP) to the content of an index register (any one of SI or DI). The default segment register may be ES or DS

eg - MOV Ax, [Bx][SI]

8) Relative based indexed.

The effective address is formed by adding an 8 or 16-bit displacement with the sum of contents of any of the base registers (Bx or BP) and any one of the index ~~reg~~ registers, in a default segment.

Eg - MOV Ax, 50H [Bx] [SI]

* Data transfer Instructions

MOV
syntax : MOV dest, src
Example : MOV Ax, Bx
Desc : Copy contents of Bx into Ax

LEA
syntax : LEA reg, mem
Example : LEA SI, [BX + DI + 10]
Desc : Load effective address into register.

XCHG
syntax : XCHG reg1, reg2
Example : XCHG Ax, Bx
Desc : Swap contents of Ax and Bx

IN
syntax : IN reg, port
Example : IN AL, 60H
Desc : Read byte from port 60H into AL.

OUT
syntax : OUT port, reg
Example : OUT 60H, AL
Desc : write AL to port 60H

PUSH
syntax : PUSH reg
Example : PUSH Ax
Desc : Push Ax onto the stack.

POP
syntax : POP reg
Example: POP AX
Desc : POP top of stack into AX

2) Arithmetic Instructions

ADD
syntax : ADD dest, src
Example : ADD AX, BX
Desc : Add BX to AX

SUB
syntax : SUB dest, src
Example : SUB AX, 1
Desc : Subtract 1 from AX

INC
syntax : INC reg
Example : INC CX
desc : Increment CX by 1

DEC
syntax : DEC reg
Example : INC Cx
Desc : Increment Cx by 1

MUL
syntax : MUL reg
Example : MUL BL
Desc : multiply AL by BL, result in AX

DIV
Syntax : DIV reg
Example : DIV CL
Desc : Divide Ax by CL

NEG
syntax : NEG reg
Example : NEG AL
Desc : Twos complemunt of AL

3) Logical Instructions

AND
syntax : AND dest, src
Example : AND AL, BL
Desc : Logical AND

OR
Syntax : OR dest, src
Example : OR Ax, OF OR H
Desc : Logical OR

XOR
Syntax : XOR AH, AH
Desc : clears AH

Not
syntax : Not reg
Example : NOT AL
Desc Invert bits of AL

TEST
Syntax: TEST reg, value
Example: TEST AL, 01H
Desc: Sets Flags based on AND result.

4) Control Transfer Instructions

JMP
Syntax: JMP address
Example: JMP 2050H
Desc: Unconditional Jump

CALL
Syntax: CALL address
Example: CALL 1234H
Desc: Call subroutine

RET
Syntax: RET
Example: RET.
Desc: Return from subroutine.

JE / JZ
Syntax: JE label
Example: JE Loop-END

WAP to transfer data from one array to another array.

```
.Model small
.data
array1 db 10H, 20H, 30H, 40H, 50H, 60H, 70H, 80H, 90H, 0AH,
array2 db 10 dup (0),
.Code
start:
        mov Ax, @data,
        mov DS, Ax,
        mov ES, Ax
        LEA SI, array1
        LEA DI, array2
        mov Cx, 10
        rep movSB
        mov ah, 40H
        INT 21H
end start
```

WAP to add the elements of an array and store the result.

```
.model small.
.data
.array db 10H, 20H, 30H, 40H, 50H, 60H, 70H, 80H, 90H, 0AH
addition db .
.code
start: mov Ax, @data
       mov DS, Ax
       mov CX, 10
       LEA SI, array1
       mov al, 00H
sum:   add al, [SI]
       mov dl, al
       inc SI
       loop sum
       mov addition dl
       mov ah, 4ch
       INT 21H
end start.
```

WAP to add two arrays element by element and store the result in third array.

```
.model small
.data
array1 db 10H, 20H, 30H, 40H, 50H, 60H, 70H, 80H, 90H, 0AH,
array2 db 10H, 12H, 15H, 20H,
array3 db ,
.code
start: mov Ax, @data
       mov Ds, Ax
       mov CX, 10
       LEA SI, array1
       LEA DI, array2
       LEA Dx, array3
       add al, [SI]
sum:   mov al, [SI]
       mov ah, [DI]
       Add al, ah
       mov [Dx], al
       inc SI
       inc DI
       inc Dx
       loop sum
       mov ah, 4ch
       int 21h
end start.
```